

第 2 周实验报告（命令行环境 / 开发环境与工具 / Debugging / Profiling）

唐朝伟一学号：25020007111

2026 年 8 月 31 日

1 第 5 题控制一个可清理的后台任务

```
mkdir q05 && cd q05
cat > worker.sh <<'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
chmod +x worker.sh

# 前台启动，stdout / stderr 分别写入两个文件
./worker.sh > stdout.log 2> stderr.log

# 按 Ctrl-Z 挂起 (SIGTSTP)，再 bg 后台继续
bg
jobs          # 确认状态为 Running

# jobs -p 只输出 PID，不手抄
jobs -p
kill -TERM $(jobs -p) # 发 SIGTERM，触发 trap 里的清理

sleep 1; jobs          # 任务已退出
cat cleanup.log        # 出现 CLEAN_EXIT
```

```
tangc@tangcw:~$ cd summerTerm/
tangc@tangcw:~/summerTerm$ mkdir q05 && cd q05
tangc@tangcw:~/summerTerm/q05$ cat > worker.sh <<'EOF'
> #!/usr/bin/env bash
> trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
> n=0
> while true; do
>   echo "$n"
>   n=$((n+1))
>   sleep 1
> done
> EOF
tangc@tangcw:~/summerTerm/q05$ ls -l worker.sh
-rw-r--r-- 1 tangc tangc 131 Sep  4 19:25 worker.sh
tangc@tangcw:~/summerTerm/q05$ chmod +x worker.sh
tangc@tangcw:~/summerTerm/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
tangc@tangcw:~/summerTerm/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
tangc@tangcw:~/summerTerm/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
tangc@tangcw:~/summerTerm/q05$ pid=$(jobs -p)
tangc@tangcw:~/summerTerm/q05$ kill -TERM "$pid"
tangc@tangcw:~/summerTerm/q05$ echo "$pid"
699
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
tangc@tangcw:~/summerTerm/q05$ wait "$pid" 2>/dev/null
tangc@tangcw:~/summerTerm/q05$ jobs
tangc@tangcw:~/summerTerm/q05$ cat cleanup.log
CLEAN_EXIT
tangc@tangcw:~/summerTerm/q05$
```

图 1: 第 5 题: 前台启动 → Ctrl-Z 挂起 → bg → jobs → jobs -p 取 PID → kill -TERM \$pid → wait \$pid 等待其结束 → cat cleanup.log 得 CLEAN_EXIT

解题感悟

1. 信号: SIGTERM(15)、SIGINT(2)、SIGTSTP(20, Ctrl-Z) 可被 trap 捕获; SIGKILL(9) 不可捕获, 所以题目禁 kill -9。
2. trap '命令' TERM INT 让进程收到信号时执行清理代码 (写 cleanup.log 再退出)。
3. 任务控制: Ctrl-Z 挂起、bg 后台继续、jobs 查看、jobs -p 取 PID (不手抄)、kill -TERM 结束。
4. 重定向: > 等价 1> 重定向 stdout, 2> 重定向 stderr。

2 第 6 题语义重构与本地开发反馈

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
```

```

from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0

```

用 VS Code 打开 q06, Pylance 语言服务器随 Python 扩展启用。演示: F12 跳转到定义、Shift+F12 查找引用、F2 重命名符号。

```

# 重命名符号 total_price -> calculate_total (F2), 定义与所有引用同步
# math_utils.py: def calculate_total(price: float, count: int) -> float:
# app.py:         from math_utils import calculate_total
#             print(calculate_total(12.5, 4))
# test_math_utils.py: assert calculate_total(12.5, 4) == 50.0

```

```

# 临时在 app.py 加入未使用的 import os, 用 ruff 发现并删除
ruff check .          # 报 F401: os imported but unused
ruff check --fix .    # 自动删除未使用 import

ruff check .          # 无报错
pytest                # 1 passed

```

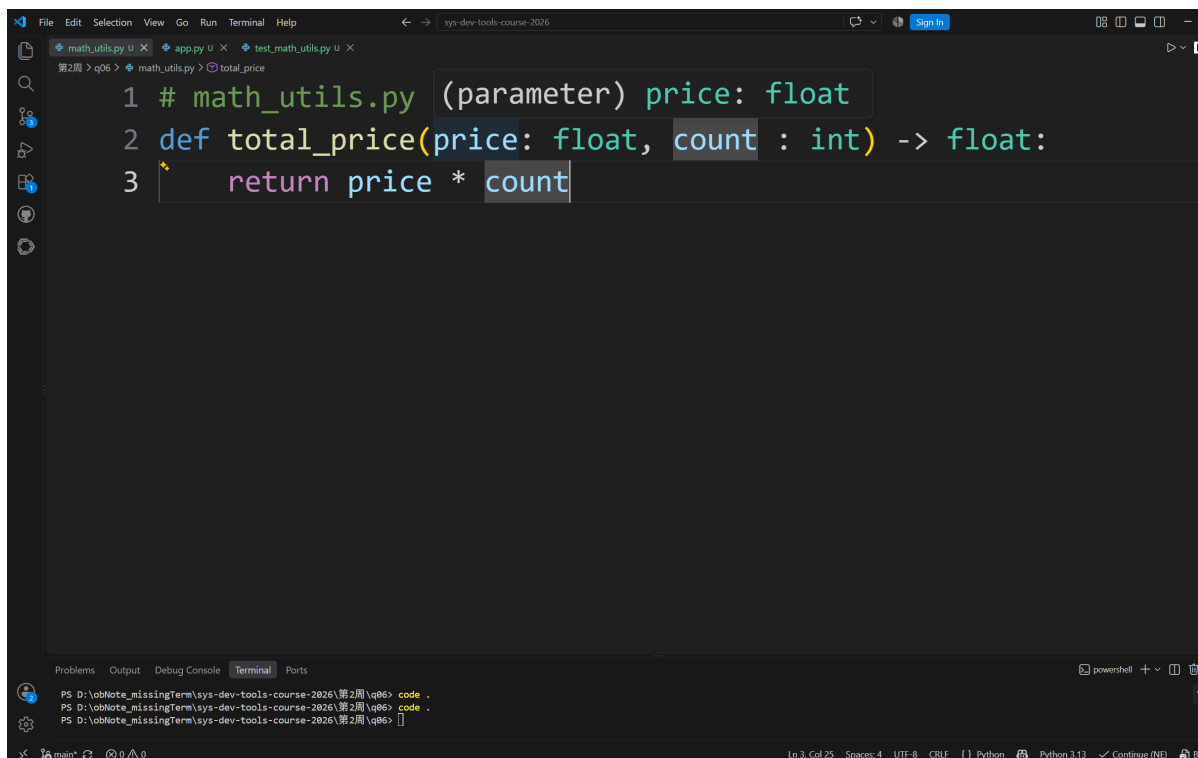


图 2: 第 6 题: 在 math_utils.py 定义处悬停参数 price, Pylance 显示 (parameter) price: float

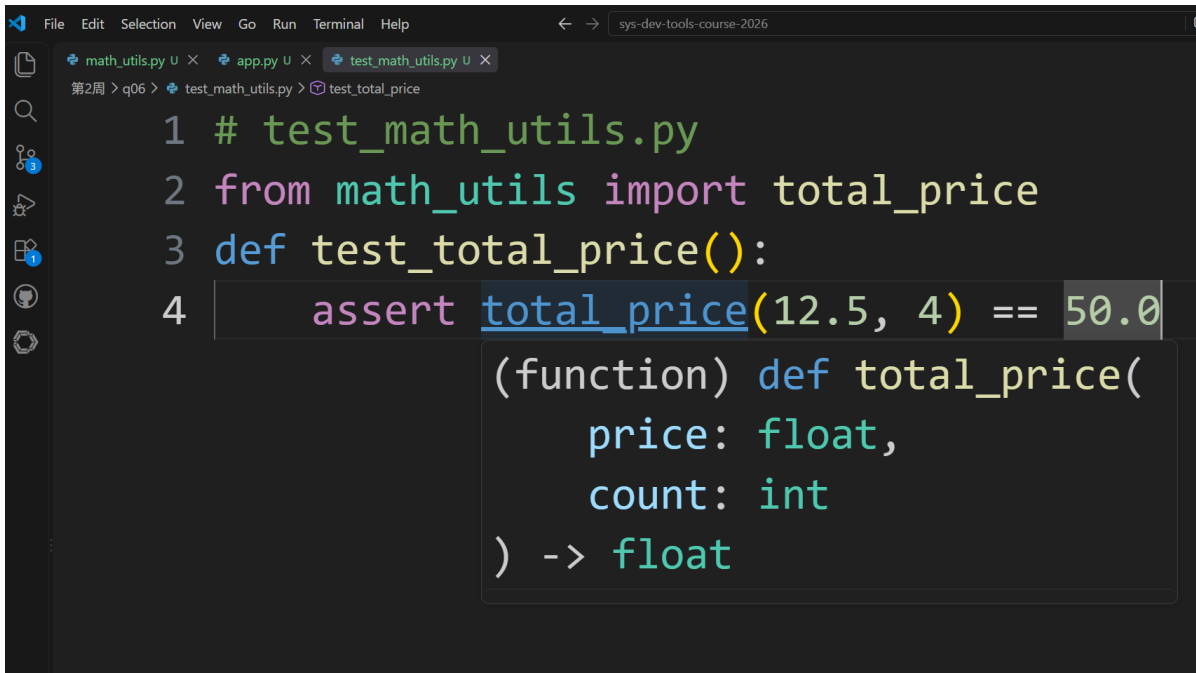


图 3: 第 6 题: Ctrl+ 点击 total_price 跳转到定义 (非 F12), 并悬停显示函数签名

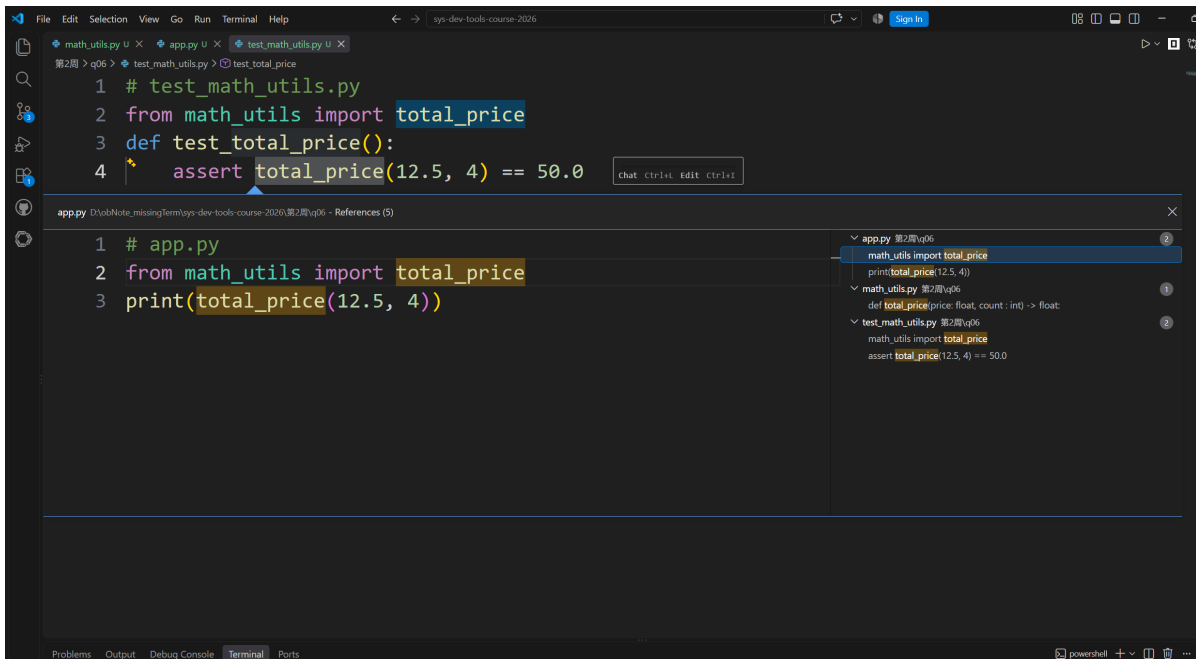


图 4: 第 6 题: Shift+F12 查找引用, 列出 total_price 的全部引用位置

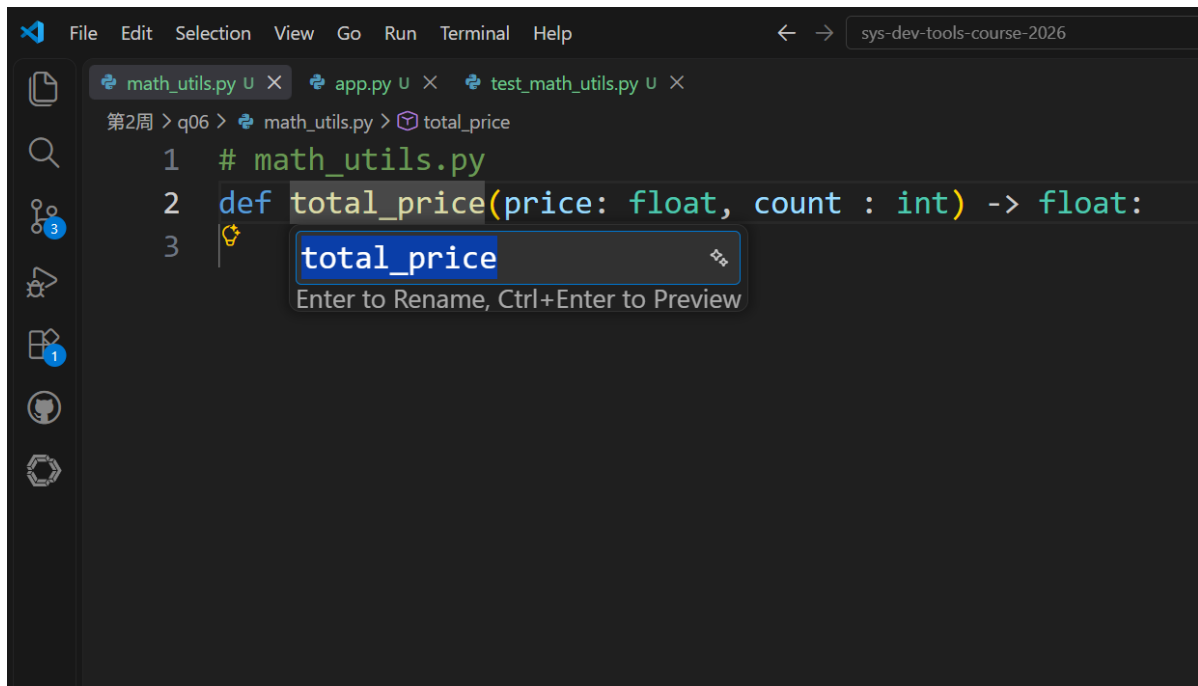


图 5: 第 6 题: F2 重命名符号, 输入新名 `calculate_total`

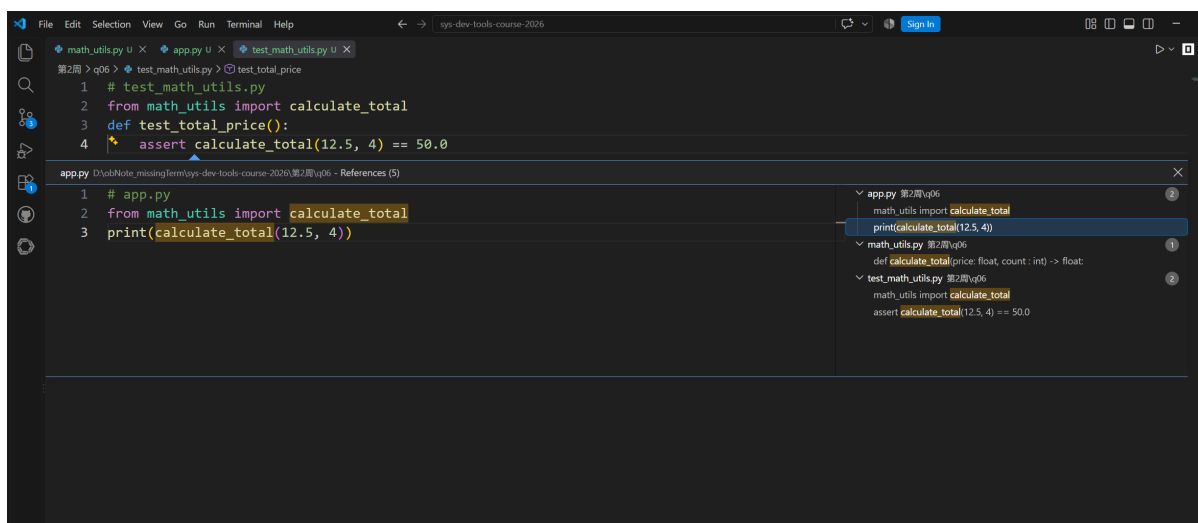


图 6: 第 6 题: 重命名后所有引用同步为 `calculate_total` (References 5)

```
File Edit Selection View Go Run Terminal Help sys-dev-tools-course-2026
```

```
math_utils.py U X app.py U X test_math_utils.py U X
```

```
第2周 > q06 > app.py
```

```
1 # app.py
2 import os
3 from math_utils import calculate_total
4 print(calculate_total(12.5, 4))
```

```
Problems Output Debug Console Terminal Ports
```

```
PS D:\obNote_missingTerm\sys-dev-tools-course-2026> cd .\第2周\q06\
PS D:\obNote_missingTerm\sys-dev-tools-course-2026\第2周\q06> python -m ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> app.py:2:1
|
1 | # app.py
2 | / import os
3 | | from math_utils import calculate_total
4 | | print(calculate_total(12.5, 4))
|
help: Organize imports
|
2 | import os
3 +
4 | from math_utils import calculate_total
5 +
6 | print(calculate_total(12.5, 4))
|
F401 [*] `os` imported but unused
--> app.py:2:8
|
1 | # app.py
2 | import os
|   ^^
3 | from math_utils import calculate_total
4 | print(calculate_total(12.5, 4))
|
help: Remove unused import: `os`
|
1 | # app.py
- import os
2 | from math_utils import calculate_total
|
I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:2:1
|
```

图 7: 第 6 题: 临时加入 import os, ruff check 报 I001 与 F401 未使用

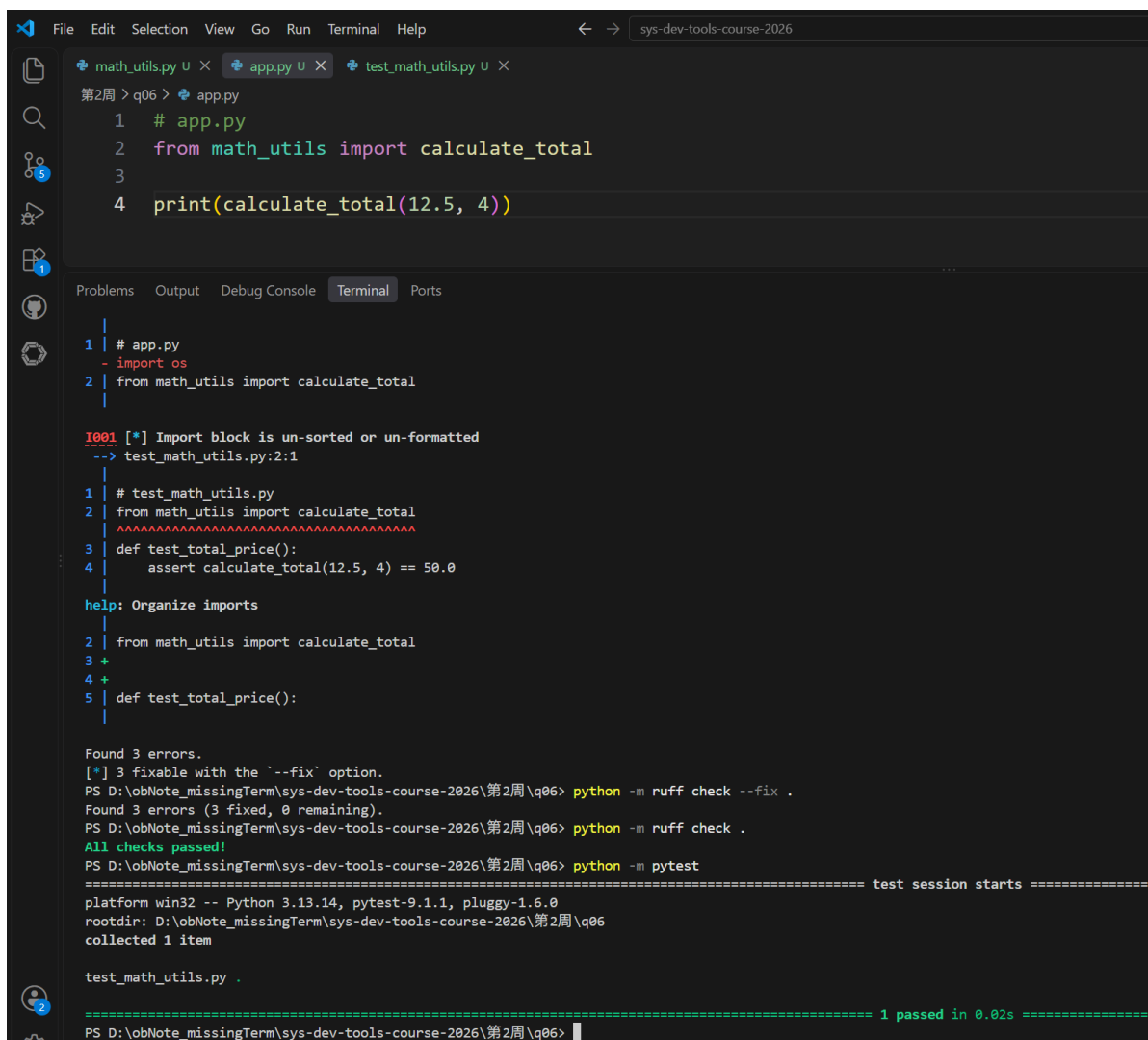


图 8: 第 6 题: ruff -fix 自动删除后 All checks passed, pytest 1 passed

解题感悟

1. 语言服务器（LSP）把编辑器与语言智能解耦，Pylance 提供跳转、补全、诊断。
2. F2 重命名是语义级的，只改”同一符号”的定义和引用，不会误伤同名不同义。
3. ruff 是极快的 linter+formatter，未使用 import 规则编号 F401; ruff check --fix 自动修复。
4. pytest 自动发现 test_*.py，用 assert 断言。

3 第 7 题用调试器定位归并排序缺陷

```

# merge_sort.py (含缺陷)
def merge(left, right):
    result = []
    i = j = 0

```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i]); i += 1
    else:
        result.append(right[i]); j += 1 # 缺陷: right[i] 应为 right[j]
return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6])) # 抛 IndexError 或结果错误

```

用 pdb 设断点观察 i、j、left[i]、right[j]:

```

python -m pdb merge_sort.py
(Pdb) b merge          # 在 merge 函数入口设断点
(Pdb) c                # 运行到断点
(Pdb) p i, j           # 观察下标
(Pdb) p left[i], right[j] # 观察比较的两个元素
(Pdb) n                # 单步, 进入 else 分支时发现 right[i] 越界/取错
(Pdb) q                # 退出

```

最小修复（不得用 sorted 替换归并排序）:

```

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 修复: right[i] -> right[j]
    return result + left[i:] + right[j:]

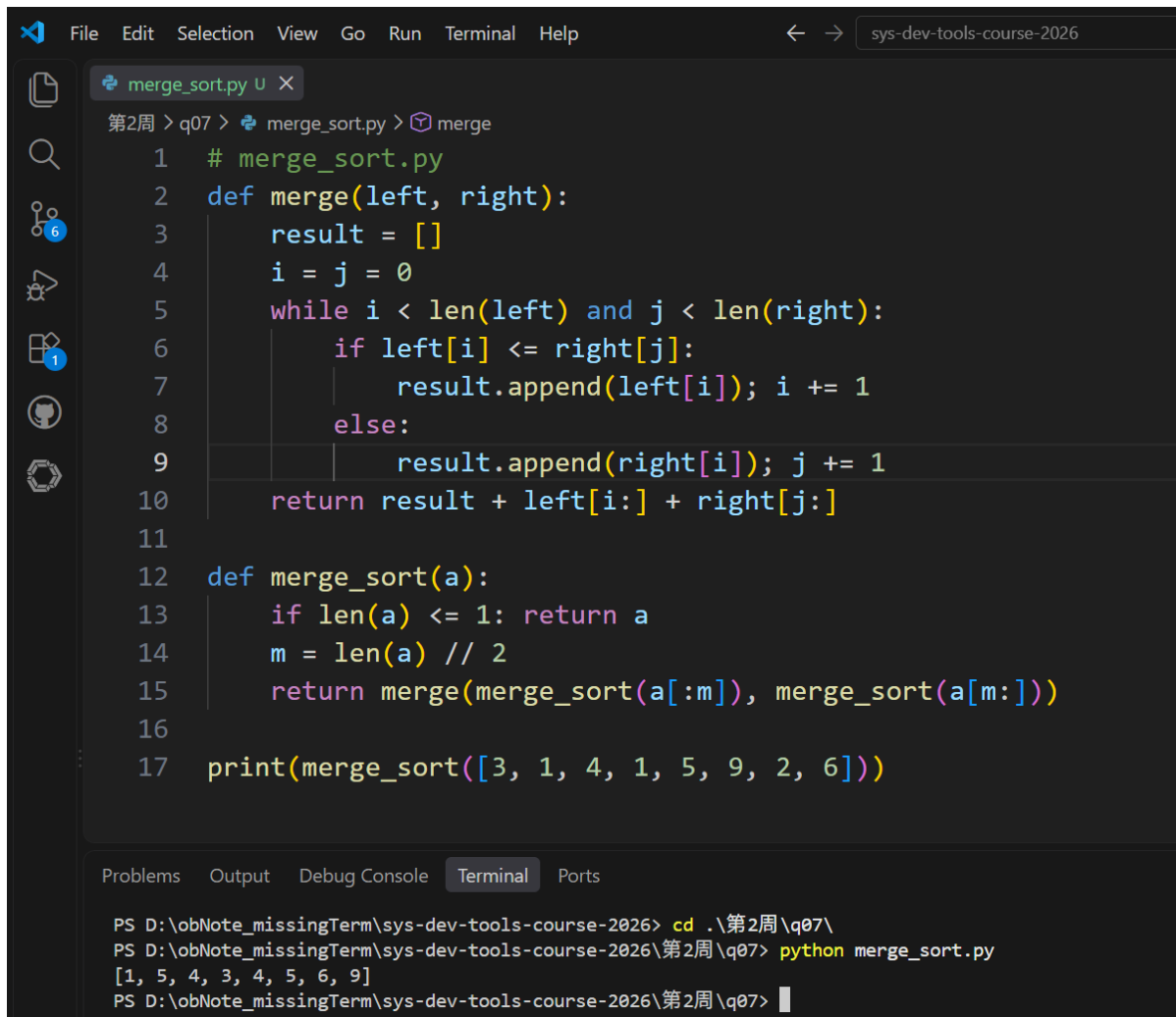
# test_merge_sort.py
from merge_sort import merge_sort
def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
def test_with_duplicates():
    assert merge_sort([5, 3, 3, 1, 2, 2]) == [1, 2, 2, 3, 3, 5]

```

```

python merge_sort.py # [1, 1, 2, 3, 4, 5, 6, 9]
pytest                # 2 passed

```



```
File Edit Selection View Go Run Terminal Help sys-dev-tools-course-2026
merge_sort.py U X
第2周 > q07 > merge_sort.py > merge
1 # merge_sort.py
2 def merge(left, right):
3     result = []
4     i = j = 0
5     while i < len(left) and j < len(right):
6         if left[i] <= right[j]:
7             result.append(left[i]); i += 1
8         else:
9             result.append(right[i]); j += 1
10    return result + left[i:] + right[j:]
11
12 def merge_sort(a):
13     if len(a) <= 1: return a
14     m = len(a) // 2
15     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
16
17 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

Problems Output Debug Console Terminal Ports
PS D:\obNote_missingTerm\sys-dev-tools-course-2026> cd .\第2周\q07\
PS D:\obNote_missingTerm\sys-dev-tools-course-2026\第2周\q07> python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 9: 第 7 题: 运行 merge_sort, 输出 [1,5,4,3,4,5,6,9] 结果错误

```

PS D:\obNote_missingTerm\sys-dev-tools-course-2026\第2周\q07> python -m pdb merge_sort.py
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(2)<module>()
-> def merge(left, right):
(Pdb) b merge
Breakpoint 1 at d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py:3
(Pdb) c
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(3)merge()
-> result = []
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(4)merge()
-> i = j = 0
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(5)merge()
-> while i < len(left) and j < len(right):
(Pdb) l
   1      # merge_sort.py
   2      def merge(left, right):
   3 B          result = []
   4              i = j = 0
   5  ->         while i < len(left) and j < len(right):
   6             if left[i] <= right[j]:
   7                 result.append(left[i]); i += 1
   8             else:
   9                 result.append(right[j]); j += 1
  10         return result + left[i:] + right[j:]
  11
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(6)merge()
-> if left[i] <= right[j]:
(Pdb) b
Num Type      Disp Enb   Where
1 breakpoint keep yes   at d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py:3
breakpoint already hit 1 time
(Pdb) p i,j,left[i],right[j],right[i]
(0, 0, 3, 1, 1)

```

图 10: 第 7 题: pdb 设断点、单步, p 打印 i,j,left[i],right[j],right[i]

```

> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(9)merge()
-> result.append(right[i]); j += 1
(Pdb) p i,j,left[i],right[j],right[i]
(1, 0, 3, 1, 4)
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(5)merge()
-> while i < len(left) and j < len(right):
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(6)merge()
-> if left[i] <= right[j]:
(Pdb) p i,j,left[i],right[j],right[i]
(1, 1, 3, 4, 4)
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(7)merge()
-> result.append(left[i]); i += 1
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(5)merge()
-> while i < len(left) and j < len(right):
(Pdb) p i,j,left[i],right[j],right[i]
*** IndexError: list index out of range
(Pdb) n
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(10)merge()
-> return result + left[i:] + right[j:]
(Pdb) p i,j,left[i],right[j],right[i]
*** IndexError: list index out of range
(Pdb) n
--Return--
> d:\obnote_missingterm\sys-dev-tools-course-2026\第2周\q07\merge_sort.py(10)merge()->[1, 4, 3, 4]
-> return result + left[i:] + right[j:]
(Pdb)

```

图 11: 第 7 题: 单步进入 else 分支, right[i] 越界抛 IndexError

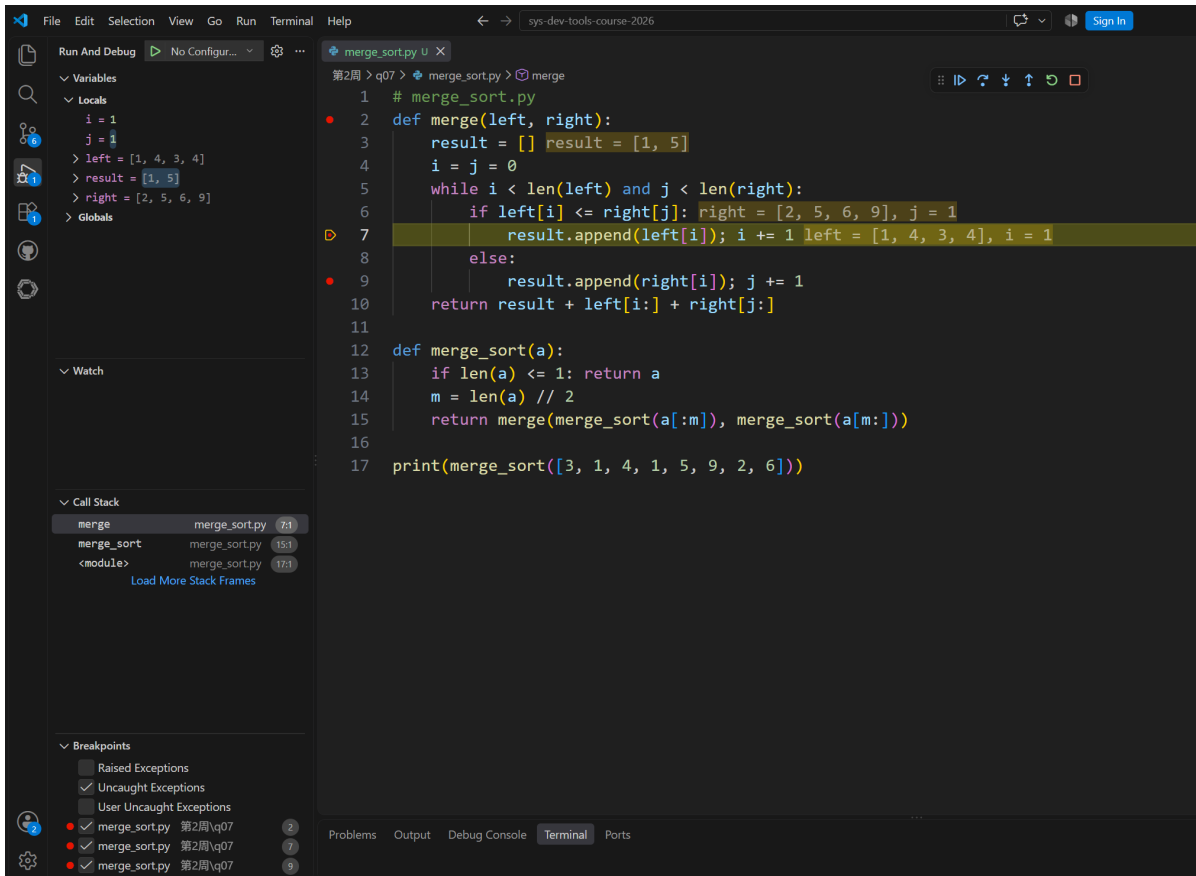


图 12: 第 7 题: VS Code 调试器观察变量与调用栈

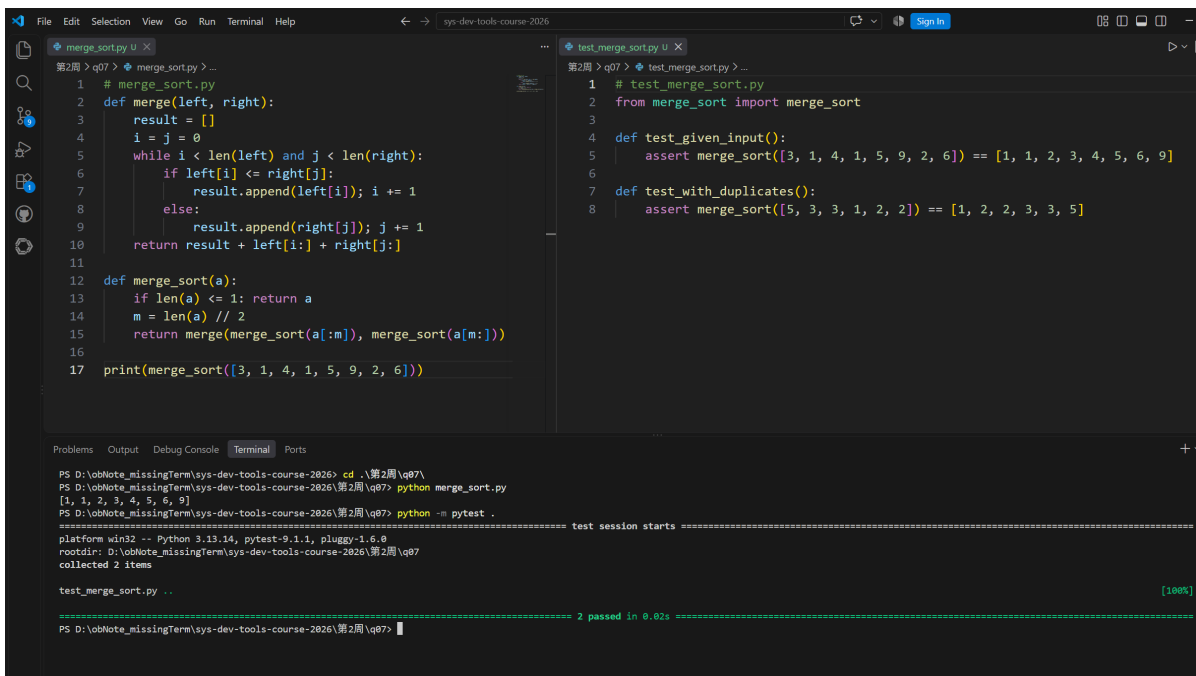


图 13: 第 7 题: 修复 right[i]→right[j] 后, pytest 两个测试均通过

解题感悟

1. 调试器让程序在运行中暂停：b 断点、c 继续、n/s 单步、p 打印变量。
2. 缺陷本质：else 分支写 `right[i]`，i 是 left 下标，应为 `right[j]`，导致越界 `IndexError`。
3. 归并排序分治：二分递归排好再 merge 合并；i 走 left、j 走 right。
4. 最小修复只改一处，不改算法结构，也不允许 `sorted`。

4 第 8 题先测量，再优化慢速词频程序

```
# generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py (原版,  $O(n^2)$ )
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

```
python generate_words.py          # 生成 3 万词
time python wordfreq.py           # 跑 2 次，记录耗时
python -m cProfile -s cumulative wordfreq.py # 找瓶颈
```

优化（不改 count、不写死 1000，用 set 去重降到 $O(n)$ ）：

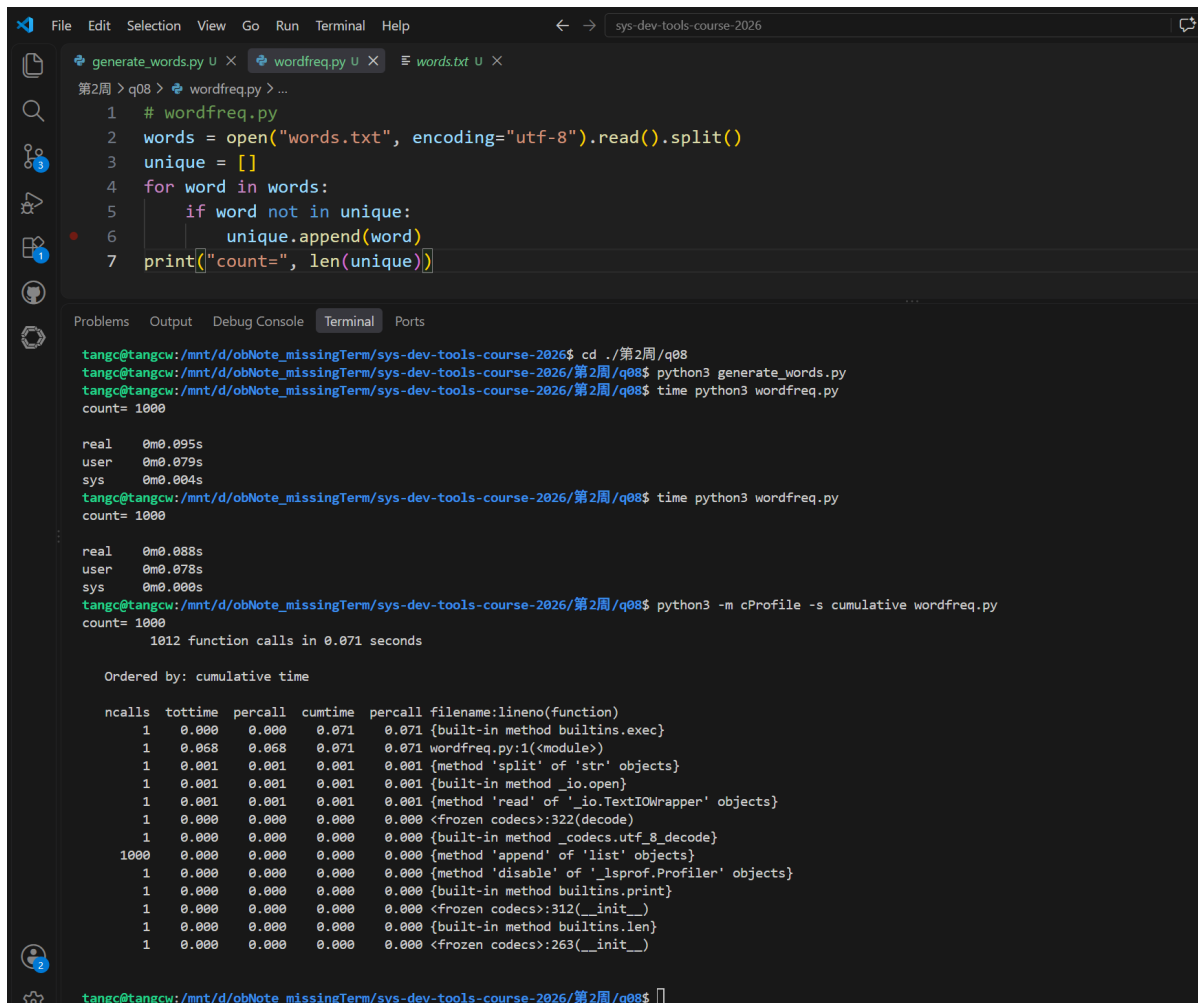
```
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)          # set 成员判断  $O(1)$ 
print("count=", len(unique)) # count=1000，与优化前一致
```

```
time python wordfreq.py # 再跑 2 次，算中位数与加速比
```

表 1: 优化前后耗时与加速比

阶段	两次耗时	中位数
优化前	0.095s、0.088s	0.0915s
优化后	0.020s、0.021s	0.0205s

加速比 $\approx 0.0915 \div 0.0205 \approx 4.5$ 倍。



The image shows a VS Code editor window with three tabs: `generate_words.py`, `wordfreq.py`, and `words.txt`. The `wordfreq.py` tab is active, displaying the following Python code:

```
1 # wordfreq.py
2 words = open("words.txt", encoding="utf-8").read().split()
3 unique = []
4 for word in words:
5     if word not in unique:
6         unique.append(word)
7 print("count=", len(unique))
```

Below the editor, the `Terminal` panel shows the execution of the script. The first run shows a total time of approximately 0.095s. The second run, using `time`, shows a total time of approximately 0.088s. The third run, using `cProfile`, shows that the script took 0.071 seconds to execute, with 1012 function calls.

```
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ cd ./第2周/q08
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ python3 generate_words.py
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ time python3 wordfreq.py
count= 1000

real    0m0.095s
user    0m0.079s
sys      0m0.004s
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ time python3 wordfreq.py
count= 1000

real    0m0.088s
user    0m0.078s
sys      0m0.000s
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.071 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.071    0.071 {built-in method builtins.exec}
1      0.068    0.068    0.071    0.071 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.001    0.001    0.001    0.001 {built-in method _io.open}
1      0.001    0.001    0.001    0.001 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
```

图 14: 第 8 题: 原版 $O(n^2)$ 词频, time 两次 (约 0.09s), cProfile 定位到循环

```
File Edit Selection View Go Run Terminal Help
generate_words.py x wordfreq.py x words.txt x
第2周 > q08 > wordfreq.py > ...
1 # wordfreq.py
2 words = open("words.txt", encoding="utf-8").read().split()
3 unique = set(words)
4 print("count=", len(unique))

Problems Output Debug Console Terminal Ports
1 0.000 0.000 0.000 0.000 {built-in method builtins.print}
1 0.000 0.000 0.000 0.000 <frozen codecs>:312(__init__)
1 0.000 0.000 0.000 0.000 {built-in method builtins.len}
1 0.000 0.000 0.000 0.000 <frozen codecs>:263(__init__)

tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ python3 wordfreq.py
count= 1000
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ time python3 wordfreq.py
count= 1000

real    0m0.020s
user    0m0.004s
sys     0m0.006s
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$ time python3 wordfreq.py
count= 1000

real    0m0.021s
user    0m0.000s
sys     0m0.009s
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/q08$
```

图 15: 第 8 题: 改用 set 去重, time 两次 (约 0.02s), count 仍为 1000

解题感悟

1. 先测量再优化: 用 `cProfile -s cumulative` 定位瓶颈, 而不是凭感觉乱改。
2. 瓶颈是 `word not in unique` 的列表成员判断 $O(n)$, 整体 $O(n^2)$; 改用 `set` 后 $O(1)$ 判断、整体 $O(n)$ 。
3. `random.seed(20260831)` 固定随机种子, 保证前后输入一致, 对比才公平。
4. 实测优化前中位数约 0.0915s、优化后约 0.0205s, 加速比约 4.5 倍; count 始终为 1000 未变。

5 课后练习 (missing-semester: Debugging & Profiling + 命令行环境)

5.1 调试 1: journalctl 查看超级用户登录记录

```
journalctl --since yesterday | grep sudo # Linux (macOS 用 log show)
sudo ls # 造几条无害记录
journalctl -u sudo --since today
```

journalctl 是 systemd 日志查看器；--since 限定时间范围，-u sudo 只看 sudo 服务日志。

```

tancg@tancgw:~$ journalctl --since yesterday | grep sudo
Sep 05 19:49:10 tancgw sudo[516]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1000)
Sep 05 19:49:10 tancgw sudo[516]: tancg : TTY=/dev/pts/0 ; PWD=/home/tancg ; USER=root ; COMMAND=/usr/bin/ls
Sep 05 19:49:10 tancgw sudo[516]: pam_unix(sudo:session): session closed for user root
Sep 05 19:49:54 tancgw sudo[541]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1000)
Sep 05 19:49:54 tancgw sudo[541]: tancg : TTY=/dev/pts/0 ; PWD=/home/tancg ; USER=root ; COMMAND=/usr/bin/ls
Sep 05 19:49:54 tancgw sudo[541]: pam_unix(sudo:session): session closed for user root
tancg@tancgw:~$ sudo ls
[sudo: authenticate] Password:
summerTerm  envv
tancg@tancgw:~$ journalctl --since yesterday | grep sudo
Sep 05 19:49:10 tancgw sudo[516]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1000)
Sep 05 19:49:10 tancgw sudo[516]: tancg : TTY=/dev/pts/0 ; PWD=/home/tancg ; USER=root ; COMMAND=/usr/bin/ls
Sep 05 19:49:10 tancgw sudo[516]: pam_unix(sudo:session): session closed for user root
Sep 05 19:49:54 tancgw sudo[541]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1000)
Sep 05 19:49:54 tancgw sudo[541]: tancg : TTY=/dev/pts/0 ; PWD=/home/tancg ; USER=root ; COMMAND=/usr/bin/ls
Sep 05 19:49:54 tancgw sudo[541]: pam_unix(sudo:session): session closed for user root
Sep 05 19:58:17 tancgw sudo[887]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1000)
Sep 05 19:58:17 tancgw sudo[887]: tancg : TTY=/dev/pts/0 ; PWD=/home/tancg ; USER=root ; COMMAND=/usr/bin/ls
Sep 05 19:58:17 tancgw sudo[887]: pam_unix(sudo:session): session closed for user root
tancg@tancgw:~$ journalctl -u sudo --since today
-- No entries --
tancg@tancgw:~$ journalctl --since today
Sep 05 13:43:05 tancgw kernel: Linux version 6.18.33.2-microsoft-standard-WSL2 (root@f1>
Sep 05 13:43:05 tancgw kernel: Command line: initrd=\initrd.img WSL_ROOT_INIT=1 panic=->
Sep 05 13:43:05 tancgw kernel: KERNEL supported cpus:
```

图 16: 课后练习 • 调试 1: journalctl 查看超级用户登录与所执行的指令 (sudo ls, COMMAND=/usr/bin/ls)

5.2 调试 2: pdb 教程

跟做 <https://github.com/spiside/pdb-tutorial> (中文版 <https://github.com/MartinLwx/pdb-tutorial>)。

调试器的价值: (1) 检查运行中程序的状态; (2) 程序投入使用前先测试; (3) 追踪执行逻辑 (比到处 print 快)。进入调试: import pdb; pdb.set_trace() 或 python -m pdb main.py。

5 个核心命令 (括号内为可省略写法):

```
l(list)  显示当前行附近 11 行源码 (ll 显示整个函数)
s(step)  单步, 会进入被调用的函数
n(ext)   单步到下一行, 不进入函数
b(reak)  设断点: b 34 / b 函数名; cl 1 删除
r(eturn) 运行到当前函数返回
```

两个技巧: 短变量名会撞命令名 (如 c 是 continue), 取值用 !c (! 表示后接 Python 语句); 用 dir(die) 看对象属性。

post-mortem（事后调试）：崩溃后用 `pdb.pm()` 回到异常抛出点；`python -m pdb 脚本.py` 遇未捕获异常会自动进入。

5.3 调试 3：shellcheck 检查脚本

```
sudo apt install shellcheck
```

原脚本存在多处问题：

```
#!/bin/sh
for f in $(ls *.m3u)
do
    grep -qi hq.*mp3 $f \
    && echo -e 'Playlist $f contains a HQ file in mp3 format'
done
```

1. `$(ls *.m3u)` 应用 `*.m3u` 直接遍历，勿用 `ls`（文件名含空格会被拆开）。
2. `$f` 应加双引号 `"$f"`。
3. `echo -e '... $f ...'` 单引号内 `$f` 不展开，应改双引号 `"$f"`。

修复后：

```
#!/bin/sh
for f in *.m3u
do
    grep -qi 'hq.*mp3' "$f" \
    && echo "Playlist $f contains a HQ file in mp3 format"
done
```

shellcheck 会报 SC2046（勿用 `ls`）、SC2086（要加引号）；编辑器装 shellcheck 插件后实时提示。

```

tangc@tangcw:~/summerTerm/w2ex$ cat > debugging3.sh <<'EOF'
> #!/bin/sh
> for f in $(ls *.m3u)
> do
>   grep -qi hq.*mp3 $f \
>   && ehco -e 'Playlist $f contains a HQ file in mp3 format'
> done
> EOF
tangc@tangcw:~/summerTerm/w2ex$ shellcheck -C auto ./debugging3.sh
auto: auto: openBinaryFile: does not exist (No such file or directory)

In ./debugging3.sh line 2:
for f in $(ls *.m3u)
      ^-----^ SC2045 (error): Iterating over ls output is fragile. Use globs.
              ^-- SC2035 (info): Use ./*glob* or -- *glob* so names with dashes won't become
options.

In ./debugging3.sh line 4:
grep -qi hq.*mp3 $f \
      ^-----^ SC2062 (warning): Quote the grep pattern so the shell won't interpret it.
              ^-- SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
grep -qi hq.*mp3 "$f" \

In ./debugging3.sh line 5:
&& ehco -e 'Playlist $f contains a HQ file in mp3 format'
              ^-- SC2016 (info): Expressions don't expand in single quotes, use double quotes
for that.

For more information:
https://www.shellcheck.net/wiki/SC2045 -- Iterating over ls output is fragi...
https://www.shellcheck.net/wiki/SC2062 -- Quote the grep pattern so the she...
https://www.shellcheck.net/wiki/SC2016 -- Expressions don't expand in singl...
tangc@tangcw:~/summerTerm/w2ex$

```

图 17: 课后练习 • 调试 3: shellcheck 检查原脚本, 报 SC2045/SC2035/SC2062/SC2086/SC2016

```

tangc@tangcw:~/summerTerm/w2ex$ vim ./debugging3.sh
tangc@tangcw:~/summerTerm/w2ex$ shellcheck -Cauto ./debugging3.sh

In ./debugging3.sh line 4:
grep -qi hq.*mp3 "$f" \
      ^-----^ SC2062 (warning): Quote the grep pattern so the shell won't interpret it.

For more information:
https://www.shellcheck.net/wiki/SC2062 -- Quote the grep pattern so the she...
tangc@tangcw:~/summerTerm/w2ex$ vim ./debugging3.sh
tangc@tangcw:~/summerTerm/w2ex$ shellcheck -Cauto ./debugging3.sh
tangc@tangcw:~/summerTerm/w2ex$ cat ./debugging3.sh
#!/bin/sh
for f in *.m3u
do
  grep -qi 'hq.*mp3' "$f" \
  && ehco -e "Playlist $f contains a HQ file in mp3 format"
done
tangc@tangcw:~/summerTerm/w2ex$

```

图 18: 课后练习 • 调试 3: 修复后 shellcheck 仅剩 SC2062 提示, cat 查看修复脚本

5.4 调试 4 (进阶): 可逆调试

阅读可逆调试白皮书, 用 rr 或 RevPDB 做可反向单步的调试例子。

```
Ubuntu
tangc@tangcw: $ cd /mnt/d/obNote_missingTerm
tangc@tangcw:/mnt/d/obNote_missingTerm$ cat ./demo.c
// demo.c
int main() {
    int x = 0;
    x = 1;
    x = 2;
    return x;
}
tangc@tangcw:/mnt/d/obNote_missingTerm$ gcc -g -o demo ./demo.c
tangc@tangcw:/mnt/d/obNote_missingTerm$ rr record ./demo
rr: Saving execution to trace directory '/home/tangc/.local/share/rr/demo-4'.
tangc@tangcw:/mnt/d/obNote_missingTerm$ rr replay
On Zen CPUs, rr will not work reliably unless you disable the hardware SpecLockMap optimization.
For instructions on how to do this, see https://github.com/rr-debugger/rr/wiki/Zen
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from /mnt/d/obNote_missingTerm/demo...
Remote debugging using 127.0.0.1:45661
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/d4/bfd49ced9d6e19bb77f23bf6872b8d270a9712.debug...
warning: BFD: warning: system-supplied DSO at 0x6ffff000 has a section extending past end of file
warning: Discarding section .replay.text which has an invalid size (27) [in module system-supplied DSO at 0x6ffff000]
0x00007aff4a59a340 in _start () from /lib64/ld-linux-x86-64.so.2
(rr) break main
Breakpoint 1 at 0x564dda4d5131: file ./demo.c, line 3.
(rr) continue
Continuing.

Breakpoint 1, main () at ./demo.c:3
3      int x = 0;
(rr) next
4      x = 1;
(rr) next
5      x = 2;
(rr) next
6      return x;
(rr) print x
```

图 19: 课后练习 · 调试 4: rr record 记录 / rr replay 进入 gdb, 设断点并单步

```
warning: Discarding section .replay.text which has an invalid size (27) [in module system-supplied DSO at 0x6ffff000]
0x00007aff4a59a340 in _start () from /lib64/ld-linux-x86-64.so.2
(rr) break main
Breakpoint 1 at 0x564dda4d5131: file ./demo.c, line 3.
(rr) continue
Continuing.

Breakpoint 1, main () at ./demo.c:3
3      int x = 0;
(rr) next
4      x = 1;
(rr) next
5      x = 2;
(rr) next
6      return x;
(rr) print x
$1 = 2
(rr) reverse-next
5      x = 2;
(rr) print x
$2 = 1
(rr) quit
A debugging session is active.

    Inferior 1 [process 749] will be detached.

Quit anyway? (y or n) y
Detaching from program: /mnt/d/obNote_missingTerm/demo, process 749
[Inferior 1 (process 749) detached]
tangc@tangcw:/mnt/d/obNote_missingTerm$
```

图 20: 课后练习 · 调试 4: reverse-next 后 print x 由 2 回到 1, 可逆调试演示成功

5.5 性能分析 1: 比较插入排序与快排

```

pip install line_profiler memory_profiler
python -m cProfile -s cumulative sorts.py # 总览
kernprof -l -v sorts.py                  # 逐行
python -m memory_profiler sorts.py       # 内存

```

插入排序瓶颈在内层 `while` 比较后移 ($O(n^2)$)；快排瓶颈在递归与 `partition`。插入排序原地 ($O(1)$ 额外空间)，教材版快排每层新建列表内存更高，故插入排序内存更优；再看原地版快排对比。附加题用 `perf` 看指令数与缓存命中。

```

Terminal
PS C:\Users\tangc> cd .\Downloads\
PS C:\Users\tangc\Downloads> python -m cProfile -s cumulative sorts.py
784397 function calls (719282 primitive calls) in 0.138 seconds

Ordered by: cumulative time

ncalls  tottime  percall  ctime   percall filename:lineno(function)
3/1     0.000    0.000    0.138   0.138  {built-in method builtins.exec}
1       0.000    0.000    0.138   0.138  sorts.py:1(module)
3       0.019    0.003    0.133   0.044  sorts.py:4(test_sorted)
77791   0.011    0.000    0.000   0.000  random.py:326(randint)
77791   0.029    0.000    0.000   0.000  random.py:295(randrange)
77791   0.019    0.000    0.029   0.000  random.py:245(randbelow_with_getrandbits)
34412/1000 0.014    0.000    0.016   0.000  sorts.py:32(quicksort_inplace)
32768/1000 0.014    0.000    0.016   0.000  sorts.py:23(quicksort)
233373   0.011    0.000    0.011   0.000  {built-in method _operator.index}
1000     0.009    0.000    0.009   0.000  sorts.py:11(insertionsort)
98371   0.005    0.000    0.005   0.000  {method 'getrandbits' of 'random.Random' objects}
77791   0.005    0.000    0.005   0.000  {method 'bit_length' of 'int' objects}
5/1      0.000    0.000    0.004   0.004  <frozen importlib._bootstrap>:1349(_find_and_load)
5/1      0.000    0.000    0.004   0.004  <frozen importlib._bootstrap>:1304(_find_and_load_unlocked)
69242   0.004    0.000    0.004   0.000  {built-in method builtins.len}
5/1      0.000    0.000    0.003   0.003  <frozen importlib._bootstrap>:911(_load_unlocked)
2/1      0.000    0.000    0.003   0.003  <frozen importlib._bootstrap_external>:1017(exec_module)
2       0.000    0.000    0.002   0.001  <frozen importlib._bootstrap_external>:1090(get_code)
10/2     0.000    0.000    0.002   0.001  <frozen importlib._bootstrap>:400(_call_with_frames_removed)
1       0.000    0.000    0.002   0.002  random.py:1(module)
4       0.000    0.000    0.002   0.000  <frozen importlib._bootstrap_external>:1211(get_data)
5       0.000    0.000    0.001   0.000  <frozen importlib._bootstrap>:1240(_find_spec)
2       0.000    0.000    0.001   0.001  <frozen importlib._bootstrap_external>:1548(find_spec)
2       0.000    0.000    0.001   0.001  <frozen importlib._bootstrap_external>:1519(_get_spec)
8       0.000    0.000    0.001   0.000  <frozen importlib._bootstrap_external>:1621(find_spec)
12       0.000    0.000    0.001   0.000  <frozen importlib._bootstrap_external>:145(_path_stat)
12       0.001    0.000    0.001   0.000  {built-in method nt.stat}
3000     0.001    0.000    0.001   0.000  {built-in method builtins.sorted}
4       0.001    0.000    0.001   0.000  {built-in method _io.open_code}
4       0.001    0.000    0.001   0.000  {method 'read' of 'io.BufferedReader' objects}
2       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:164(_path_isfile)
2       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:155(_path_is_mode_type)
5       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap>:416(__enter__)
40       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:101(_path_join)
2       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:1230(path_stats)
3       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap>:597(exec_module)
3       0.000    0.000    0.000   0.000  {built-in method _imp.exec_builtins}
2       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:779(_compile_bytecode)
2       0.000    0.000    0.000   0.000  {built-in method marshal.loads}
5       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap>:806(module_from_spec)
4       0.000    0.000    0.000   0.000  {method '__exit__' of '_io._IOBase' objects}
5       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap>:304(acquire)
1       0.000    0.000    0.000   0.000  bisect.py:1(module)
5       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap>:733(_init_module_attrs)
4       0.000    0.000    0.000   0.000  <frozen importlib._bootstrap_external>:509(cache_from_source)

```

```

C:\Users\tangc> Downloads > sorts.py
4 def test_sorted(fn, iters=1000):
5     for i in range(iters):
6         l = [random.randint(0, 100) for i in range(0, random.randint(0, 50))]
7         assert fn(l) == sorted(l)
8         # print(fn.__name__, fn(l))
9
10
11 def insertionsort(array):
12
13     for i in range(len(array)):
14         j = i - 1
15         v = array[i]
16         while j >= 0 and v < array[j]:
17             array[j+1] = array[j]
18             j -= 1
19         array[j+1] = v
20     return array
21
22
23 def quicksort(array):
24     if len(array) <= 1:
25         return array
26     pivot = array[0]
27     left = [i for i in array[1:] if i < pivot]
28     right = [i for i in array[1:] if i >= pivot]
29     return quicksort(left) + [pivot] + quicksort(right)
30
31
32 def quicksort_inplace(array, low=0, high=None):
33     if len(array) <= 1:
34         return array
35     if high is None:
36         high = len(array) - 1
37     if low >= high:
38         return array
39
40     pivot = array[high]
41     j = low - 1
42     for i in range(low, high):
43         if array[i] <= pivot:
44             j += 1
45             array[i], array[j] = array[j], array[i]
46     array[high], array[j+1] = array[j+1], array[high]
47     quicksort_inplace(array, low, j)
48     quicksort_inplace(array, j+2, high)
49     return array
50
51
52 if __name__ == '__main__':
53     for fn in [quicksort, quicksort_inplace, insertionsort]:
54         test_sorted(fn)
55

```

图 21: 课后练习 · 性能 1: cProfile -s cumulative 总览 sorts.py

```
Terminal
Timer unit: 1e-06 s

PS C:\Users\tangc\Downloads> python -m kernprof -l -v sorts.py
Wrote profile results to 'sorts.py.lprof'
Timer unit: 1e-06 s

Total time: 0.128801 s
File: sorts.py
Function: insertionsort at line 10

Line #      Hits          Time    Per Hit   % Time  Line Contents
=====
10           1             0.0         0.0      0.0      @profile
11           1             0.0         0.0      0.0      def insertionsort(array):
12
13          25005          4325.2         0.2      3.4      for i in range(len(array)):
14          25005          4234.1         0.2      3.3          j = i-1
15          25005          4378.1         0.2      3.4          v = array[i]
16          226443          41372.6         0.2     32.1          while j >= 0 and v < array[j]:
17          281438          36888.8         0.2     28.0              array[j+1] = array[j]
18          281438          33767.2         0.1     26.2              j -= 1
19          25005          4437.5         0.2      3.4              array[j+1] = v
20           1             0.0         0.0      0.0      return array

Total time: 0.0432633 s
File: sorts.py
Function: quicksort at line 22

Line #      Hits          Time    Per Hit   % Time  Line Contents
=====
22           1             0.0         0.0      0.0      @profile
23           1             0.0         0.0      0.0      def quicksort(array):
24          33944          7006.7         0.2     16.2      if len(array) <= 1:
25          17472          2962.0         0.2      6.8          return array
26          16472          2791.9         0.2      6.5      pivot = array[0]
27          16472          8294.6         0.5     19.2      left = [i for i in array[1:] if i < pivot]
28          16472          7899.2         0.5     18.3      right = [i for i in array[1:] if i >= pivot]
29          16472          14388.9         0.9     33.1      return quicksort(left) + [pivot] + quicksort(right)

Total time: 0.117931 s
File: sorts.py
Function: quicksort_inplace at line 31

Line #      Hits          Time    Per Hit   % Time  Line Contents
=====
31           1             0.0         0.0      0.0      @profile
32           1             0.0         0.0      0.0      def quicksort_inplace(array, low=0, high=None):
33          33236          6085.4         0.2      5.2      if len(array) <= 1:
34           49             8.8         0.2      0.0          return array
35          33187          5585.7         0.2      4.7      if high is None:
36           951           194.3         0.2      0.2          high = len(array)-1
37          33187          5606.4         0.2      4.8      if low >= high:
```

```
sorts.py
C:\Users\tangc\Downloads> sorts.py > quicksort_inplace

1 import random
2
3
4 def test_sorted(fn, iters=1000):
5     for i in range(iters):
6         l = [random.randint(0, 100) for i in range(100)]
7         assert fn(l) == sorted(l)
8         # print(fn.__name__, fn(l))
9
10 @profile
11 def insertionsort(array):
12
13     for i in range(len(array)):
14         j = i-1
15         v = array[i]
16         while j >= 0 and v < array[j]:
17             array[j+1] = array[j]
18             j -= 1
19         array[j+1] = v
20     return array
21
22 @profile
23 def quicksort(array):
24     if len(array) <= 1:
25         return array
26     pivot = array[0]
27     left = [i for i in array[1:] if i < pivot]
28     right = [i for i in array[1:] if i >= pivot]
29     return quicksort(left) + [pivot] + quicksort(right)
30
31 @profile
32 def quicksort_inplace(array, low=0, high=None):
33     if len(array) <= 1:
34         return array
35     pivot = array[low]
36     left = low
37     right = high
38     while left < right:
39         while array[left] < pivot:
40             left += 1
41         while array[right] >= pivot:
42             right -= 1
43         array[left], array[right] = array[right], array[left]
44     array[left], array[low] = array[low], array[left]
45     quicksort_inplace(array, left, right)
46     return array
```

图 22: 课后练习 • 性能 1: kernprof -l -v 逐行分析插入排序 / 快排 / 原地快排

Terminal				
PS C:\Users\tangc\Downloads> python -m memory_profiler sorts.py				
Filename: sorts.py				
Line #	Mem usage	Increment	Occurrences	Line Contents
=====				
10	36.410 MiB	36.410 MiB	1000	@profile
11				def insertionsort(array):
12				
13	36.410 MiB	0.000 MiB	25719	for i in range(len(array)):
14	36.410 MiB	0.000 MiB	24719	j = i-1
15	36.410 MiB	0.000 MiB	24719	v = array[i]
16	36.410 MiB	0.000 MiB	225570	while j >= 0 and v < array[j]:
17	36.410 MiB	0.000 MiB	200851	array[j+1] = array[j]
18	36.410 MiB	0.000 MiB	200851	j -= 1
19	36.410 MiB	0.000 MiB	24719	array[j+1] = v
20	36.410 MiB	0.000 MiB	1000	return array
Filename: sorts.py				
Line #	Mem usage	Increment	Occurrences	Line Contents
=====				
22	37.277 MiB	-25534.914 MiB	33586	@profile
23				def quicksort(array):
24	37.277 MiB	-25572.227 MiB	33586	if len(array) <= 1:
25	37.277 MiB	-13173.125 MiB	17293	return array
26	37.277 MiB	-12399.109 MiB	16293	pivot = array[0]
27	37.277 MiB	-94316.074 MiB	123779	left = [i for i in array[1:] if i < pivot]
28	37.277 MiB	-94318.914 MiB	123779	right = [i for i in array[1:] if i >= pivot]
29	37.277 MiB	-12404.000 MiB	16293	return quicksort(left) + [pivot] + quicksort(right)
Filename: sorts.py				
Line #	Mem usage	Increment	Occurrences	Line Contents
=====				
31	36.410 MiB	36.406 MiB	33030	@profile
32				def quicksort_inplace(array, low=0, high=None):
33	36.410 MiB	0.000 MiB	33030	if len(array) <= 1:
34	36.410 MiB	0.000 MiB	46	return array
35	36.410 MiB	0.000 MiB	32984	if high is None:
36	36.410 MiB	0.000 MiB	954	high = len(array)-1
37	36.410 MiB	0.000 MiB	32984	if low >= high:
38	36.410 MiB	0.000 MiB	16969	return array
39				
40	36.410 MiB	0.000 MiB	16015	pivot = array[high]
41	36.410 MiB	0.000 MiB	16015	j = low-1
42	36.410 MiB	0.000 MiB	120723	for i in range(low, high):
43	36.410 MiB	0.000 MiB	104708	if array[i] <= pivot:
44	36.410 MiB	0.000 MiB	53688	j += 1
45	36.410 MiB	0.000 MiB	53688	array[i], array[j] = array[j], array[i]
46	36.410 MiB	0.004 MiB	16015	array[high], array[j+1] = array[j+1], array[high]

图 23: 课后练习 · 性能 1: memory_profiler 对比内存（教材版快排每层新建列表，内存更高）

5.6 性能分析 2: pycallgraph 画斐波那契调用图

```
pip install pycallgraph
dot -V # 能运行 dot 即已装 graphviz
```

```
#!/usr/bin/env python
def fib0(): return 0
def fib1(): return 1
s = """def fib{ }(): return fib{ }() + fib{ }()"""
if __name__ == '__main__':
```

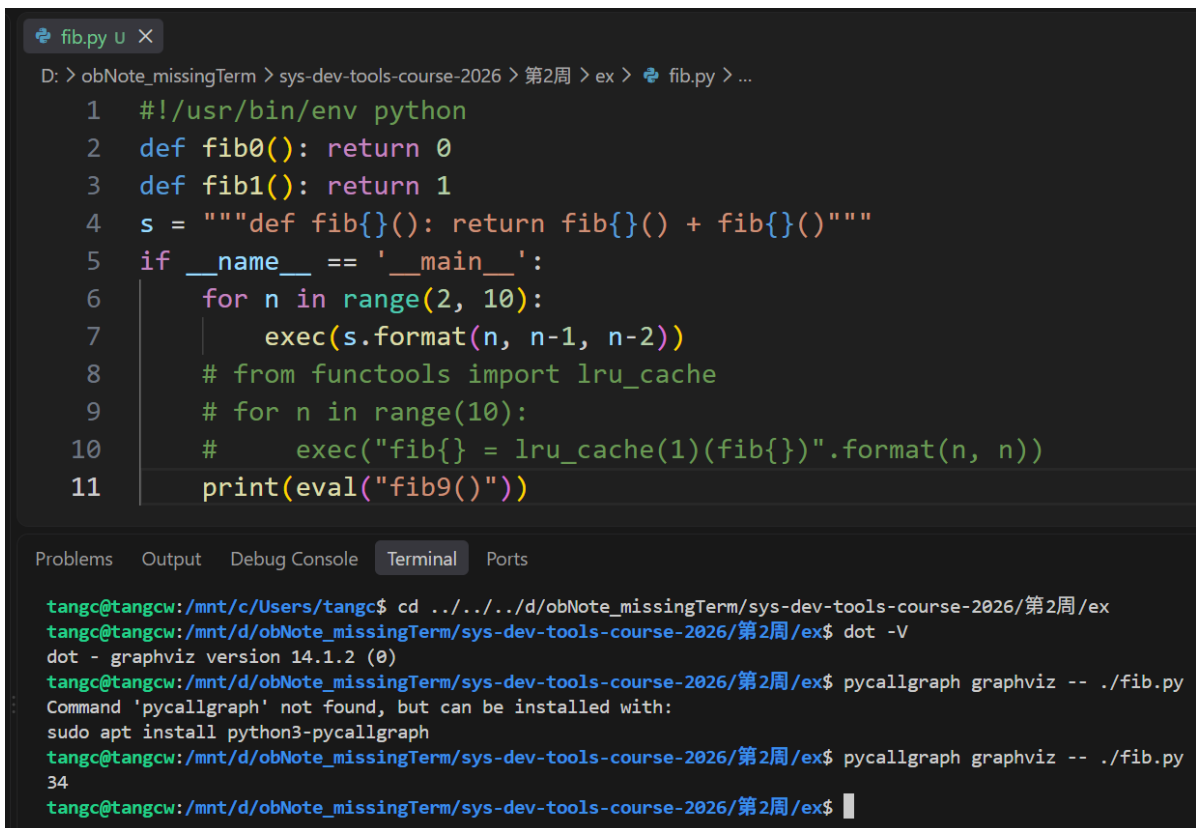
```

for n in range(2, 10):
    exec(s.format(n, n-1, n-2))
# from functools import lru_cache
# for n in range(10):
#     exec("fib{} = lru_cache(1)(fib{}).format(n, n))
print(eval("fib9()"))

```

```
pycallgraph graphviz -- ./fib.py # 生成 pycallgraph.png
```

原始递归版调用 fib9 时呈指数级重复：fib0 被调用 21 次、fib1 34 次，整个计算共约 109 次函数调用（调用图密集）；放开 lru_cache 后，fib0~fib9 每个函数因缓存只真正算一次，共 10 次调用（调用图稀疏）——即记忆化。



```

fib.py u x
D: > obNote_missingTerm > sys-dev-tools-course-2026 > 第2周 > ex > fib.py > ...
1  #!/usr/bin/env python
2  def fib0(): return 0
3  def fib1(): return 1
4  s = """def fib{}(): return fib{}() + fib{}()"""
5  if __name__ == '__main__':
6      for n in range(2, 10):
7          exec(s.format(n, n-1, n-2))
8      # from functools import lru_cache
9      # for n in range(10):
10         #     exec("fib{} = lru_cache(1)(fib{}).format(n, n))
11     print(eval("fib9()"))

```

```

tangc@tangcw:/mnt/c/Users/tangc$ cd ../../../../d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$ dot -V
dot - graphviz version 14.1.2 (0)
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$ pycallgraph graphviz -- ./fib.py
Command 'pycallgraph' not found, but can be installed with:
sudo apt install python3-pycallgraph
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$ pycallgraph graphviz -- ./fib.py
34
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$

```

图 24: 课后练习 · 性能 2: pycallgraph 未装时提示先安装, fib.py 源码与 dot 已装

```
fib.py u X
D: > obNote_missingTerm > sys-dev-tools-course-2026 > 第2周 > ex > fib.py > ...
1  #!/usr/bin/env python
2  def fib0(): return 0
3  def fib1(): return 1
4  s = """def fib{ }(): return fib{ }() + fib{ }()"""
5  if __name__ == '__main__':
6      for n in range(2, 10):
7          exec(s.format(n, n-1, n-2))
8          from functools import lru_cache
9          for n in range(10):
10             exec("fib{ } = lru_cache(1)(fib{ })".format(n, n))
11             print(eval("fib9()"))

Problems Output Debug Console Terminal Ports
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$ pycallgraph -o pycallgraph2.png graphviz -- ./fib.py
usage: pycallgraph [options] OUTPUT_TYPE [output_options] -- SCRIPT.py [ARG ...]
pycallgraph: error: argument output: invalid choice: 'pycallgraph2.png' (choose from graphviz, gephi)
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$ pycallgraph graphviz -o pycallgraph2.png -- ./fib.py
34
tangc@tangcw:/mnt/d/obNote_missingTerm/sys-dev-tools-course-2026/第2周/ex$
```

图 25: 课后练习 · 性能 2: pycallgraph graphviz -o 生成调用图, fib9 输出 34

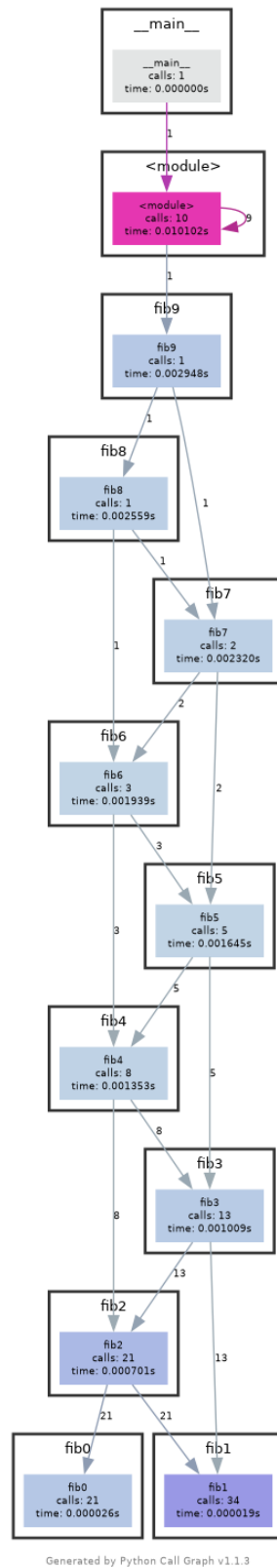


图 26: 课后练习 • 性能 2: 未加 lru_cache, fib0 被重复调用极多次 (调用图密集)

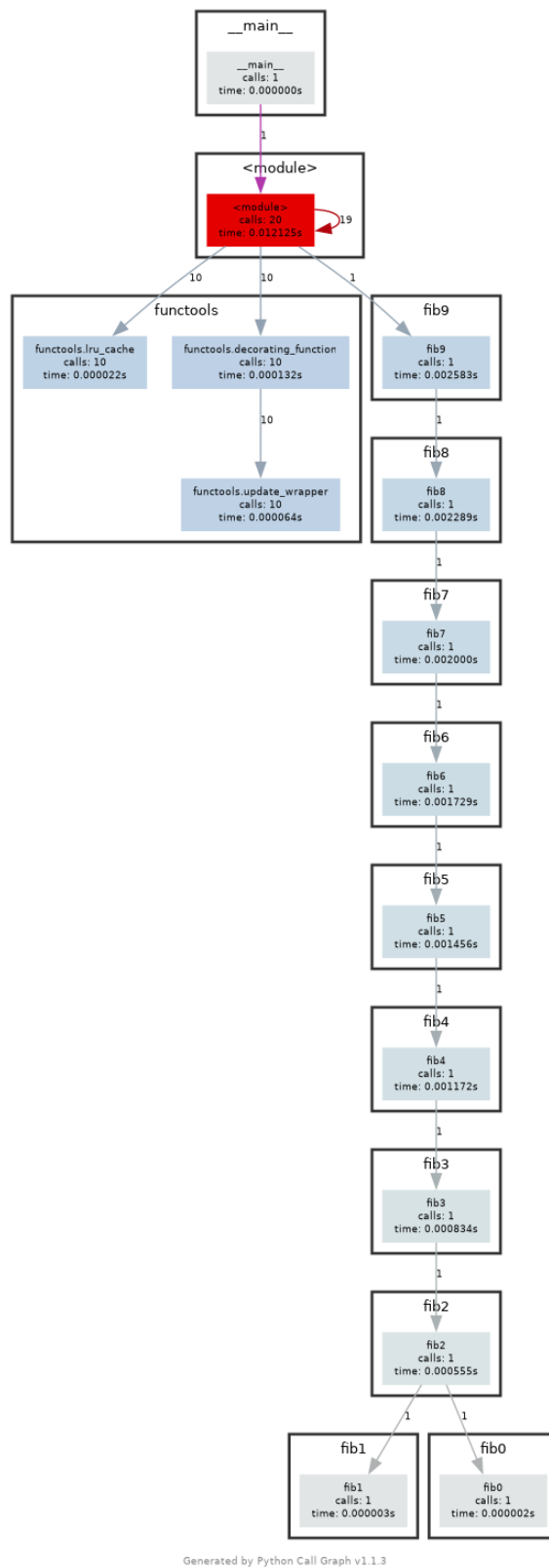


图 27: 课后练习 • 性能 2: 放开 lru_cache 后, 每个 fibN 只算一次, 调用图稀疏

5.7 性能分析 3: lsof 查端口占用

```
python -m http.server 4444 &
lsof | grep LISTEN # 或 lsof -i :4444
```

```
kill <PID>
```



```
wsl: 检测到 localhost 代理配置, 但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
tangc@tangcw:~$ python3 -m http.server 4444 &
[1] 1013
tangc@tangcw:~$ Serving HTTP on 0.0.0.0 port 4444 (http://0.0.0.0:4444/) ...

wsl: 检测到 localhost 代理配置, 但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
tangc@tangcw:~$ lsof | grep LISTEN
python3    1013          tangc      3u      IPv4  35451      0t0  TCP
*:4444 (LISTEN)
tangc@tangcw:~$ kill 1013
tangc@tangcw:~$ lsof | grep LISTEN
tangc@tangcw:~$
```

图 28: 课后练习 · 性能 3: python -m http.server 4444 起服务, lsof 查 PID, kill 后端口释放

5.8 性能分析 4: stress + htop + taskset 限制 CPU

```
sudo apt install stress htop
stress -c 3 # 3 个 worker, htop 看约 3 核满载
htop
taskset --cpu-list 0,2 stress -c 3 # 绑到核 0、2, 再看 htop
```

stress -c 3 开 3 个 worker 线程各跑 CPU 死循环, 不加限制时内核把它们调度到不同核, 能占满 3 个核; taskset --cpu-list 0,2 设置的是进程的 CPU 亲和性, 把整个进程 (连同 3 个线程) 限制在核 0、2 上, 于是 3 个线程只能挤在这 2 个核里, 最多看到 2 核满载、第 3 个线程排队等——所以不是 3 个核满载。

附加题: 用 cgroups 限制 stress -m 内存。

```
sudo apt install -y cgroup-tools
sudo mkdir -p /sys/fs/cgroup/limit_mem
echo 128M | sudo tee /sys/fs/cgroup/limit_mem/memory.max # 上限 128M
stress -m 4 --vm-bytes 64M & # 4 个 worker 各 64M, 共想占 256M
echo $! | sudo tee /sys/fs/cgroup/limit_mem/cgroup.procs # 把进程丢进 cgroup
# 结果: 被限制在 128M 内, 内存不再暴涨
```

cgroups (control groups) 是内核按进程组限 CPU/内存/IO 的机制, v2 里写 memory.max 即设内存上限, 超过会回收甚至 OOM。

5.9 性能分析 5 (进阶): Wireshark 抓 curl 报文

curl ipinfo.io 发起请求, Wireshark 抓包, 过滤器填 http 只看 HTTP 报文。

5.10 任务控制 1: pgrep / pkill

```
sleep 10000
# Ctrl-Z 挂起
bg
pgrep -af sleep      # -a 显示完整命令行, -f 匹配完整命令行
pkill -f 'sleep 10000' # 按命令行匹配结束, 无需手抄 pid
```

5.11 任务控制 2: pidwait 函数

```
pidwait() {
    while kill -0 "$1" 2>/dev/null; do
        sleep 1
    done
}

sleep 60 &
pidwait $! # $! 是最近一个后台任务 pid
ls
```

`wait` 只能等当前 shell 的子进程, 跨会话无效; `kill -0` 不发信号、只检测进程是否存在 (存在返回 0), 配 `sleep 1` 轮询避免烧 CPU。

5.12 终端多路复用 tmux

完成 <https://www.hamvocke.com/blog/a-quick-and-easy-guide-to-tmux/> 教程, 掌握分屏 (`Ctrl-b %` / `Ctrl-b "`)、脱离会话 (`Ctrl-b d`)、重连 (`tmux attach`) 等。

6 提交记录

课程要求将每次报告、练习、代码上传到个人 GitHub 并保留 commit 记录 (禁止单次全量提交), 故本报告采用分批提交: 笔记 / 报告源码 / 报告 PDF / q05~q08 / 课后练习各占一次提交。

- 仓库链接: <https://github.com/silksnow/sys-dev-tools-course-2026>
- 提交记录截图: